

Expense Management API

Deployment Guide & Change Checklist (Free Hosting)

1. What you are building

This guide deploys the stack on free tiers of three services: **Neon** for the PostgreSQL database, **Render** for the FastAPI backend, and **Vercel** for the React frontend. All three have free tiers that never require a credit card.

- **Neon** - free Postgres database (connection string becomes DATABASE_URL)
- **Render** - free web service running FastAPI + Alembic migrations
- **Vercel** - free static hosting for the built React app

Good news: the codebase has already been made deployment-ready. Configuration is environment-variable driven, the schema is managed by Alembic migrations, a health check endpoint exists, and the dashboard works without Redis. You mostly need to fill in real values, not rewrite code.

2. Prepare the code

1. **Step 1.** Open a terminal in the project folder (Expense-managment-API).
2. **Step 2.** Create a GitHub repository (or push to the existing one) and upload this project. All new files (Dockerfile, alembic/, tests/, .env.example, .github/) must be committed.
3. **Step 3.** Generate a strong secret key with:

```
python -c "import secrets; print(secrets.token_urlsafe(64))"
```

Copy the output - you will paste it into Render's environment variables.

3. Database - create a free Postgres on Neon

1. **Step 1.** Go to neon.tech and sign up with GitHub or Google.
2. **Step 2.** Click **Create a project** (region closest to you, PostgreSQL 16 or newer).
3. **Step 3.** Copy the connection string shown. Use the one labelled **psycopg/SQLAlchemy** (starts with postgresql://). Example:

```
postgresql://user:password@ep-something.region.aws.neon.tech/finance_db?sslmode=require
```

Keep this string safe. It replaces the current localhost value.

4. Backend - deploy the API on Render

1. **Step 1.** Go to render.com and sign up (GitHub sign-in).
2. **Step 2.** Click **New +** -> **Web Service** -> connect your GitHub repository.
3. **Step 3.** Render detects the repo. For Build Command set:
4. **Step 4.** For Start Command set (single worker - free tier has ~512 MB RAM):
5. **Step 5.** Create the service, then open the **Environment** tab and add the variables below.

```
pip install -r requirements.txt
```

```
alembic upgrade head && gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 1 --bind 0.0.0.0:$PORT
```

Required environment variables (must be set):

Variable	Value
APP_ENV	production
DATABASE_URL	Your Neon connection string from step 3
SECRET_KEY	The 64-character secret you generated in step 2
CORS_ORIGINS	Your Vercel frontend URL, e.g. https://your-app.vercel.app (set after step 5)

Optional environment variables (set if you use these features):

Variable	Value
GOOGLE_CLIENT_ID	Google OAuth client ID (if using Google sign-in)
MAIL_USERNAME	Gmail address used for sending emails
MAIL_PASSWORD	Gmail App Password (not your Gmail password)
MAIL_FROM	The same Gmail address
REDIS_URL	Leave empty - dashboard falls back to direct DB reads

Skip Redis on the free tier: the dashboard already degrades gracefully when Redis is unavailable.

5. Frontend - deploy on Vercel

- Step 1.** Go to vercel.com and sign in with GitHub.
- Step 2.** Click **Add New** -> **Project** -> select the repo.
- Step 3.** Set **Root Directory** to **frontend** (the React app lives there).
- Step 4.** Framework auto-detects as Vite. Build command **npm run build**, output **dist**.
- Step 5.** Add an environment variable `VITE_GOOGLE_CLIENT_ID` (if using Google sign-in).
- Step 6.** Deploy, then note your URL: `https://your-app.vercel.app`

Connect the frontend to the API

The frontend calls the API through the relative path `/api`. On Vercel, add a **vercel.json** file in the frontend/ folder so `/api/*` requests are forwarded to Render. Create `frontend/vercel.json` with:

```
{
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://YOUR-RENDER-URL.onrender.com/:path*"
    }
  ]
}
```

Replace `YOUR-RENDER-URL` with your actual Render service URL. After adding this file, push the change - Vercel redeploys automatically.

6. Google OAuth (only if you use Google sign-in)

- Step 1.** Go to console.cloud.google.com -> your project -> APIs & Services -> Credentials.
- Step 2.** Edit your OAuth 2.0 Client ID.
- Step 3.** Under **Authorized JavaScript origins** add: `https://your-app.vercel.app`
- Step 4.** Under **Authorized redirect URIs** add: `https://your-render-service.onrender.com`

5. **Step 5.** Make sure `GOOGLE_CLIENT_ID` is set in both Render (backend) and Vercel (frontend) env vars.

7. Verify the deployment

1. **Step 1.** Open `https://your-render-service.onrender.com/health` - you should see:
2. **Step 2.** Open your Vercel URL and complete a full registration + login + add a transaction.
3. **Step 3.** Check the Render logs tab - the migration should have run (alembic upgrade head).

```
{
  "status": "ok",
  "app": "Finance API",
  "env": "production",
  "database": "ok"
}
```

8. Free-tier limitations (important)

- Render free web services **sleep after 15 minutes** of inactivity; the first request after sleep is slow.
- Neon free databases pause when idle; unpause happens on the first connection.
- Vercel has no cold start but inherits the backend's cold start on the first API call.
- Google sign-in emails (OTP/welcome) use Gmail SMTP - requires a Gmail App Password.

9. What YOU must change - full checklist

- **SECRET_KEY** - generate a real 64-char value (Render env var).
- **DATABASE_URL** - replace the localhost Postgres URL with your Neon URL.
- **APP_ENV=production** and **DEBUG=false** on Render.
- **CORS_ORIGINS** - set to your real Vercel frontend URL.
- **frontend/vercel.json** - create it with your Render URL in the rewrite.
- **VITE_GOOGLE_CLIENT_ID** - set on Vercel if using Google sign-in.
- **Google Cloud** - add the Vercel origin and Render redirect to your OAuth client.
- **Gmail App Password** - create one for `MAIL_PASSWORD` (do not paste your normal password).
- **Start command** - use `-w 1` on Render's free tier (512 MB RAM).
- **Port** - gunicorn must bind to `0.0.0.0:$PORT` (Render injects `$PORT`).
- **Test data** - the local dev DB stays local; Neon starts empty, so register a fresh account.

10. Alternative - run everything on one free VM (Oracle Cloud)

If you prefer not to split across three platforms, Oracle Cloud Free Tier provides an always-free ARM VM (4 vCPUs / 24 GB RAM). On that VM you run the project as-is with docker compose - no sleeping, no expiry. It needs a credit card to verify your account, a VM setup, and a domain or IP-based HTTPS, which is more work.

```
cp .env.example .env # edit .env with real values
docker compose up -d --build
```

End of guide.